

---

# Computing View

## Documento de Apresentação do Sistema

---

Sistema de visão computacional que monitora uma sala (câmera ao vivo ou arquivo de vídeo) e responde, em tempo real, a três perguntas: quem está na sala?, quem está usando o celular? e por quanto tempo? — entregando ao final relatórios de frequência e de uso de celular em CSV e PDF.

### Três pilares

- **Detecção de uso de celular** — pose + detecção do aparelho + postura.
- **Presença e identidade** — reconhecimento facial e tempo de permanência.
- **Relatórios** — frequência por aluno e ocupação da sala (CSV + PDF).

Arquitetura em camadas SOLID, ligadas por injeção de dependência. 85 testes automatizados que não dependem de pesos YOLO nem de câmera.

# 1. Como o sistema se comporta

---

## 1.1. Detecção de uso de celular — três sinais combinados

A decisão "está usando o celular" usa três sinais, com precedência:

1. **Proximidade pulso ↔ celular (primário).** Dois modelos YOLO11 rodam juntos: a pose localiza pessoas e o esqueleto (ombros, cotovelos, **pulsos**) e a detecção localiza o **celular**. A pessoa é marcada quando um pulso cai dentro de um raio proporcional ao seu tamanho — regra invariante à distância da câmera. Elimina os falsos positivos (celular perto do pé/cintura ou de quem está ao lado).
2. **Contenção (fallback).** Só quando nenhum pulso confiável está perto, e apenas para pessoas com pulsos não visíveis, usa-se a geometria antiga (IoU + fração do celular dentro da pessoa). Pulso visível e longe significa "não está segurando" — é o ganho de precisão.
3. **Postura (autônomo).** Mesmo sem o YOLO ver o aparelho (celular escuro/oculto), a postura típica já marca o uso: mão erguida à frente do tronco + cotovelo flexionado + cabeça inclinada para baixo. Generaliza além do que o dataset COCO enxerga.

## 1.2. Estabilidade do estado (sem "piscar")

Para o rótulo não oscilar (laranja ↔ verde) a cada frame quando a pessoa se mexe, o sistema:

- **Rastreia** cada pessoa entre frames (**ByteTrack/BoT-SORT**), com `track_id` estável;
- aplica **suavização temporal com histerese** — vota numa janela deslizante e usa dois limiares (liga em 0.6, só desliga em 0.35; a "zona morta" entre eles mata o tremido);
- mantém um **grace period**: o box sobrevive a oclusões/sumiços breves.

## 1.3. Feedback visual e adaptação de hardware

- Pessoa usando celular → caixa **verde** (com alvo/crosshair); demais → **laranja**. Braços em ciano, mãos em magenta; a mão que segura o aparelho em vermelho.
- Alunos identificados exibem o **nome**; uso confirmado só pela postura recebe o rótulo "Usando Celular (postura)".
- O dispositivo é escolhido sozinho: **CUDA → MPS (Apple) → CPU**. Inferência em `imgsz=960` por padrão; modo leve para máquinas fracas.

## 2. Funcionalidades por modo de uso

| Comando                                     | Para quê serve                                               |
|---------------------------------------------|--------------------------------------------------------------|
| <code>python3 -m src.main</code>            | Webcam em tempo real (modo padrão).                          |
| <code>source_file video &lt;arq&gt;</code>  | Processa um arquivo de vídeo.                                |
| <code>demo</code>                           | Cena sintética (sem YOLO/câmera) — abre a janela na hora.    |
| <code>attendance</code>                     | Liga presença + reconhecimento facial + relatórios.          |
| <code>enroll data/students</code>           | Matrícula: gera a galeria de embeddings dos alunos.          |
| <code>ui</code>                             | Painel de controle desktop (matricular, iniciar, relatório). |
| <code>eval-images &lt;pasta&gt;</code>      | Valida a detecção em fotos reais (calibra limiares).         |
| <code>save / no-display / max-frames</code> | Gravar saída anotada, rodar headless, limitar frames.        |

### 2.1. Sistema de presença e relatórios

Ao final de uma sessão com `attendance/ui`, são gerados em `data/reports/<fonte>_<timestamp>/:`

| Arquivo                            | Conteúdo                                                                                                                                                                                                              |
|------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>frequencia.csv / .pdf</code> | Por aluno: presente?, tempo total, <b>tempo usando o celular</b> , % do tempo no celular, 1º/último avistamento, % da sessão. Ausentes aparecem explicitamente; anônimos que usam o celular entram como "Pessoa #ID". |
| <code>ocupacao.csv / .pdf</code>   | Série temporal (pessoas, identificados, usando celular) + pico e média de ocupação.                                                                                                                                   |

O tempo é medido em segundos de vídeo (frame / fps), tornando os relatórios reproduzíveis a partir de um arquivo.

### 3. Arquitetura (camadas)

| Módulo          | Responsabilidade                                                                   |
|-----------------|------------------------------------------------------------------------------------|
| config.py       | Configuração global imutável (limiares, classes, cores); tudo via CVUM_*.          |
| video_source.py | Strategy + Factory para a fonte (webcam/arquivo); expõe o FPS real.                |
| detector.py     | YOLO (pose + detecção) + regra de negócio (pulso, contenção, postura) + tracking.  |
| temporal.py     | Suavização temporal/histerese por track_id.                                        |
| visualizer.py   | Desenho de caixas, esqueleto, rótulos e HUD.                                       |
| text_render.py  | Texto Unicode (acentos) via Pillow/TrueType.                                       |
| eval_images.py  | Validação da detecção contra imagens estáticas.                                    |
| main.py         | Loop principal — orquestra tudo por injeção de dependência.                        |
| attendance/     | Presença: enrollment, face_recognizer, geometry, attendance, session, reports, ui. |

### 4. Testes

Suíte com **85 testes** (python3 -m pytest -v). Princípio central: os testes **mockam YOLO, OpenCV e InsightFace** — não baixam pesos nem exigem câmera/GPU, então rodam rápido em CI.

| Arquivo                 | Qtd. | Cobertura                                                          |
|-------------------------|------|--------------------------------------------------------------------|
| test_detector.py        | 29   | Geometria, regra pulso↔celular, device, parsing de pose, tracking. |
| test_posture.py         | 7    | Score de postura e marcação autônoma.                              |
| test_temporal.py        | 6    | Histerese, grace period, evicção de tracks.                        |
| test_session.py         | 8    | Dwell time, pontes em sumiços curtos, ocupação.                    |
| test_attendance.py      | 8    | Presença, identidade por track, movimentação.                      |
| test_face_recognizer.py | 6    | Match por cosseno + cache de identidade.                           |
| test_enrollment.py      | 7    | Galeria de embeddings + matrícula.                                 |
| test_reports.py         | 3    | Geração de CSV/PDF.                                                |
| test_eval_images.py     | 5    | Harness de validação por imagens.                                  |
| test_demo.py            | 6    | Cena sintética + detector roteirizado.                             |

## 5. Novas melhorias (ainda não enviadas)

---

O commit inicial entregava apenas detecção básica (modelo único + contenção). Tudo abaixo é trabalho local, ainda não enviado.

1. **Pipeline de pose (esqueleto)** — decisão por proximidade do pulso, muito mais precisa que a caixa inteira.
2. **Análise de postura** — sinal independente da aparência do aparelho.
3. **Rastreamento entre frames** (ByteTrack/BoT-SORT) com `track_id` estável.
4. **Suavização temporal com histerese + grace period** — fim do "piscar".
5. **Sistema de presença completo** — reconhecimento facial, dwell time, movimentação e relatórios CSV + PDF (frequência e ocupação).
6. **Painel de controle desktop (ui) com a Application em thread.**
7. **Validador por imagens** (eval-images) para calibrar sem câmera.
8. **Seleção automática de dispositivo** (CUDA → MPS → CPU) e `imgsz` ajustável.
9. **Limiar de confiança separado para o celular** (CVUM\_PHONE\_CONF) — recupera o aparelho sem afrouxar a detecção de pessoas.
10. **FPS real da fonte de vídeo** — relatórios em segundos reproduzíveis.
11. **Configuração 100% por ambiente** (CVUM\_\*) sobre uma Config imutável.

## 6. Correções de falhas (ainda não enviadas)

---

1. **Falsos positivos por proximidade de caixa** (celular perto do pé/cintura ou de quem está ao lado) → regra pulso↔celular.
2. **Rótulo "piscando"** (laranja↔verde) → tracking + histerese + grace.
3. **Celular escuro/borrado perdido** pelos modelos nano/small → modelo medium + `imgsz=960` + limiar próprio + reforço de raio pela postura.
4. **Janela preta no macOS** (Tk 8.5) → painel equivalente em OpenCV (`cv_panel.py`), com botões e atalhos; backend via CVUM\_UI\_BACKEND.
5. **Acentos saindo como "?"/caixas** (`cv2.putText` é só ASCII) → texto via Pillow/TrueType, em uma única passada por frame.
6. **Auto-instalação do lap em runtime** (ByteTrack) → fixado no `requirements.txt`.
7. **Comparação de array NumPy como booleano** no recorte facial → comparação explícita com `None`.
8. **Imports pesados protegidos** (InsightFace/Pillow/fpdf2/torch) — testes e geometria não exigem tê-los; CSV sai mesmo sem `fpdf2`.

